

런타임 에러 (Runtime Errors)



성공적인 컴파일, 그러나 멈춰버린 프로그램
— 실행 중 오류 완벽 해부

오늘 습득할 디버깅 스킬 (학습 목표)



개념 확립: 컴파일 에러 vs 런타임 에러 차이 이해



원인 분석: 0으로 나누기 오류의 수학적/논리적 원인 파악



메모리 이해: 배열 범위 초과(Out of Bounds) 인식 및 수정



로그 해독: 에러 메시지를 읽고 정확한 원인 추적



해결책 적용: 방어적 코딩(Defensive Coding)을 통한 에러 사전 예방

에러의 두 가지 얼굴: 문법 vs 논리

컴파일 에러 (Compile Error)



특징: 문법(Syntax) 오류.

결과: 번역 실패. 실행 파일 자체가
아예 만들어지지 않음.

런타임 에러 (Runtime Error)



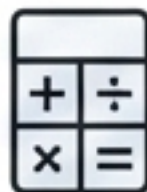
특징: 논리(Logic) 및 환경 오류.

결과: 컴파일은 무사히 통과하지만,
실행 중 프로그램이 강제 종료됨.

가장 흔한 런타임 에러의 2대 원인



런타임 에러 발생!
(프로그램 강제 종료)



1. 0으로 나누기 (Division by Zero)

수학적으로 불가능한 연산을
컴퓨터에게 지시했을 때 발생.



2. 배열 범위 초과 (Array Out of Bounds)

나에게 허락되지 않은 메모리
공간을 몰래 훑쳐보려 할 때 발생.

원인 분석 1: 0으로 나누기 (Division by Zero)



```
int x = 10;  
int y = 0;
```

```
printf("%d", x / y); // 문법적 결함은 없음
```

따라해보세요: 컴퓨터는 0으로 나누는 방법을 모릅니다. 코드가 실행되는 순간, 즉시 크래시(Crash)가 발생합니다!

- > 컴파일 성공
- > 실행 중...
- > 프로그램 강제 종료됨 (Floating point exception)



해결책: 방어적 코딩 (Defensive Coding)으로 막아내기

에러가 터지기 전에 미리 조건문(if)으로 차단하는 안전한 습관

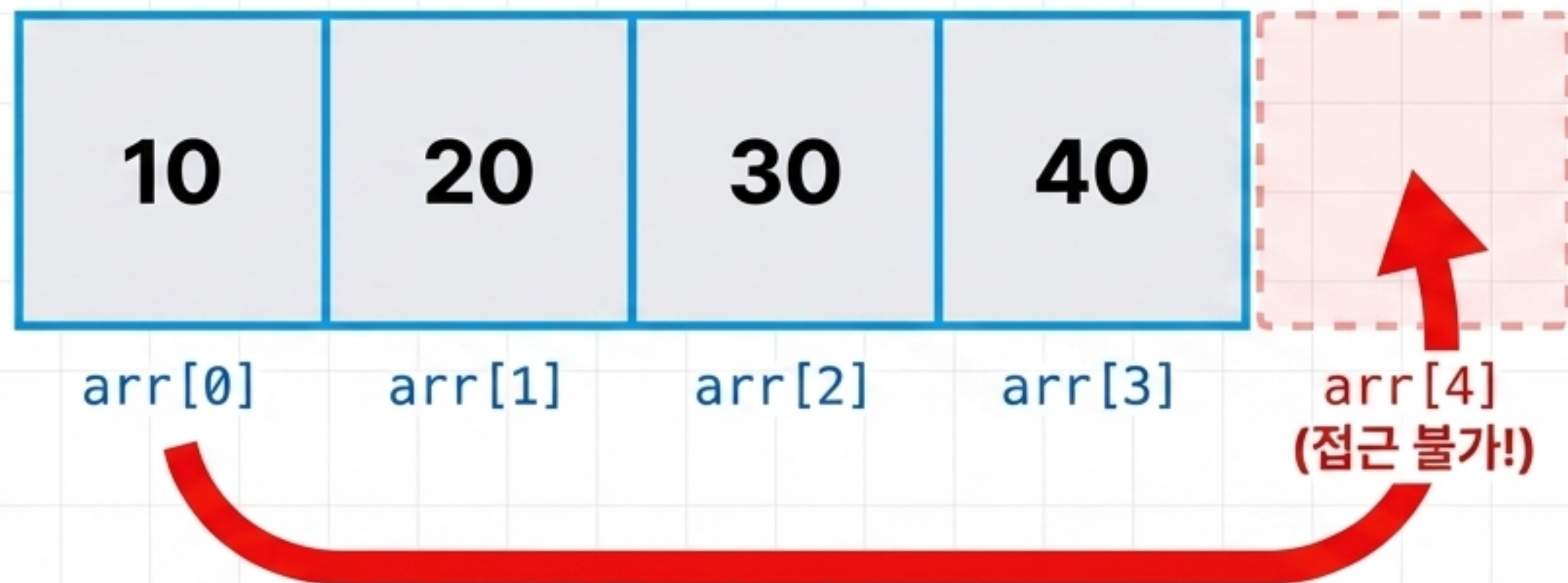


```
if (b != 0) {  
    printf("%d", a / b);  
} else {  
    printf("나눌 수 없음");  
}
```


원인 분석 2: 배열 범위 초과 (Array Out of Bounds)

```
int arr[4] = {10, 20, 30, 40};
```

- 크기가 N인 배열의 허용 인덱스는 항상 0 부터 N-1 까지.
- 할당되지 않은 메모리에 강제로 접근하면 발생하는 치명적 오류.



에러 메시지: Segmentation fault
(메모리 접근 오류)

직접 풀어보세요 (Lab 1): 숨은 에러 찾기

난이도: 쉬움

문제 코드

```
int x = 8, y = 0;  
printf("%d", x / y); // 왜 여기서 멈출까요?
```



원인: 0으로 나누기 시도



정답: 나누기 연산 전에 방어막을 세워야 합니다.

```
if (y != 0) {  
    printf("%d", x / y);  
}
```


직접 풀어보세요 (Lab 2): 배열 범위와 입력 검사

●●● [Mission A: 배열 범위 확인] (난이도: 쉬움)

```
int arr[4] = {1, 2, 3, 4};
```

질문: 올바른 인덱스 범위를 쓰고, 모든 값을 출력하세요.

해답: 인덱스는 0, 1, 2, 3 입니다.
(출력: 1 2 3 4)

●●● [Mission B: 방어적 나눗셈] (난이도: 보통)

미션: scanf로 두 정수(a, b)를 입력받아 나눕니다. b가 0이면 '오류'를 출력하세요.

```
scanf("%d %d", &a, &b);  
if (b != 0) { ...정상 계산... }  
else { printf("오류"); }
```

● 해답 핵심 로직

핵심 요약 (Core Takeaways)

- [✓] 런타임 에러 정의: 컴파일 성공 후 실행 중에 돌발적으로 발생하는 오류.
- [✓] 2대 주요 원인: 0으로 나누기, 배열 범위 초과.
- [✓] 방어적 코딩: 실행 전 `if (b != 0)` 등으로 오류 조건을 사전 차단하는 습관.
- [✓] 배열 인덱스 규칙: 크기 N 배열의 인덱스는 무조건 `0 ~ N-1` 까지만 존재.
- [✓] Segmentation fault: 잘못된 메모리 영역을 건드렸을 때 나타나는 대표적 에러 메시지.